

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score

df = pd.read_csv("chicago_taxi_trips_1000.csv")

```

```
df
```

	Trip ID	Taxi ID	Trip Start Timestamp	Trip End Timestamp	Trip Seconds \
0	100000	TX2679	2024-08-21 01:23:00	2024-08-21 01:35:36	756
1	100001	TX9928	2024-01-12 02:22:00	2024-01-12 02:33:23	683
2	100002	TX5552	2024-04-11 12:52:00	2024-04-11 12:57:26	326
3	100003	TX1711	2024-02-04 00:25:00	2024-02-04 00:55:56	1856
4	100004	TX2139	2024-01-29 22:33:00	2024-01-29 23:15:41	2561
..	...	...	...	...	...
995	100995	TX1455	2024-03-15 10:30:00	2024-03-15 10:41:21	681
996	100996	TX4927	2024-09-27 12:39:00	2024-09-27 13:34:07	3307
997	100997	TX1847	2024-11-02 06:01:00	2024-11-02 06:22:44	1304
998	100998	TX2754	2024-08-20 16:18:00	2024-08-20 17:02:33	2673
999	100999	TX2649	2024-02-04 00:41:00	2024-02-04 00:51:16	616

	Trip Miles	Pickup Community Area	Dropoff Community Area	Fare
0	0.99	70	12	5.63
1	4.76	54	29	14.67
2	15.30	20	28	39.97
3	2.39	59	69	8.99

8.47					
4	6.22		6	30	18.18
6.29					
..	...		...	...	...
...					
995	2.32		70	11	8.82
4.37					
996	3.05		29	39	10.57
4.54					
997	14.66		36	11	38.43
8.07					
998	17.41		19	37	45.03
1.11					
999	10.14		63	3	27.59
3.61					

	Tolls	Extras	Trip Total	Payment Type	Company
0	0.0	1	9.38	No Charge	Flash Cab
1	0.0	1	20.72	No Charge	Medallion
2	2.5	2	46.07	Mobile	Flash Cab
3	5.0	2	24.46	Cash	Taxi Affiliation
4	0.0	1	25.47	Mobile	Flash Cab
..	...	...	...	...	...
995	5.0	1	19.19	No Charge	Flash Cab
996	0.0	1	16.11	Credit Card	Medallion
997	2.5	2	51.00	Credit Card	Flash Cab
998	5.0	0	51.14	Mobile	Yellow Cab
999	0.0	3	34.20	No Charge	Flash Cab

[1000 rows x 15 columns]

df.head()

	Trip ID	Taxi ID	Trip Start Timestamp	Trip End Timestamp	Trip Seconds \
0	100000	TX2679	2024-08-21 01:23:00	2024-08-21 01:35:36	756
1	100001	TX9928	2024-01-12 02:22:00	2024-01-12 02:33:23	683
2	100002	TX5552	2024-04-11 12:52:00	2024-04-11 12:57:26	326
3	100003	TX1711	2024-02-04 00:25:00	2024-02-04 00:55:56	1856
4	100004	TX2139	2024-01-29 22:33:00	2024-01-29 23:15:41	2561

	Trip Miles	Pickup Community Area	Dropoff Community Area	Fare	Tips \
0	0.99		70	12	5.63
2.75					

1	4.76	54	29	14.67
5.05				
2	15.30	20	28	39.97
1.60				
3	2.39	59	69	8.99
8.47				
4	6.22	6	30	18.18
6.29				

	Tolls	Extras	Trip Total	Payment Type	Company
0	0.0	1	9.38	No Charge	Flash Cab
1	0.0	1	20.72	No Charge	Medallion
2	2.5	2	46.07	Mobile	Flash Cab
3	5.0	2	24.46	Cash	Taxi Affiliation
4	0.0	1	25.47	Mobile	Flash Cab

df.tail()

	Trip ID	Taxi ID	Trip Start Timestamp	Trip End Timestamp	Trip Seconds \
995	100995	TX1455	2024-03-15 10:30:00	2024-03-15 10:41:21	681
996	100996	TX4927	2024-09-27 12:39:00	2024-09-27 13:34:07	3307
997	100997	TX1847	2024-11-02 06:01:00	2024-11-02 06:22:44	1304
998	100998	TX2754	2024-08-20 16:18:00	2024-08-20 17:02:33	2673
999	100999	TX2649	2024-02-04 00:41:00	2024-02-04 00:51:16	616

	Trip Miles	Pickup Community Area	Dropoff Community Area	Fare	Tips \
995	2.32	70	11	8.82	4.37
996	3.05	29	39	10.57	4.54
997	14.66	36	11	38.43	8.07
998	17.41	19	37	45.03	1.11
999	10.14	63	3	27.59	3.61

	Tolls	Extras	Trip Total	Payment Type	Company
995	5.0	1	19.19	No Charge	Flash Cab
996	0.0	1	16.11	Credit Card	Medallion
997	2.5	2	51.00	Credit Card	Flash Cab
998	5.0	0	51.14	Mobile	Yellow Cab
999	0.0	3	34.20	No Charge	Flash Cab

```
df.shape
(1000, 15)

df.info()

<class 'pandas.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 15 columns):
#   Column                                Non-Null Count  Dtype  
---  -
0    Trip ID                             1000 non-null   int64   
1    Taxi ID                             1000 non-null   str      
2    Trip Start Timestamp                1000 non-null   str      
3    Trip End Timestamp                  1000 non-null   str      
4    Trip Seconds                        1000 non-null   int64   
5    Trip Miles                          1000 non-null   float64  
6    Pickup Community Area               1000 non-null   int64   
7    Dropoff Community Area              1000 non-null   int64   
8    Fare                                1000 non-null   float64  
9    Tips                                1000 non-null   float64  
10   Tolls                               1000 non-null   float64  
11   Extras                              1000 non-null   int64   
12   Trip Total                          1000 non-null   float64  
13   Payment Type                        1000 non-null   str      
14   Company                             1000 non-null   str      
dtypes: float64(5), int64(5), str(5)
memory usage: 117.3 KB

df.isnull().sum()

Trip ID          0
Taxi ID          0
Trip Start Timestamp  0
Trip End Timestamp  0
Trip Seconds     0
Trip Miles       0
Pickup Community Area  0
Dropoff Community Area  0
Fare             0
Tips             0
Tolls            0
Extras           0
Trip Total       0
Payment Type     0
Company          0
dtype: int64

df.describe()
```

	Trip ID	Trip Seconds	Trip Miles	Pickup Community Area
\				
count	1000.000000	1000.000000	1000.000000	1000.000000
mean	100499.500000	2011.099000	10.446050	39.211000
std	288.819436	962.585243	5.580252	22.281179
min	100000.000000	306.000000	0.530000	1.000000
25%	100249.750000	1142.750000	5.877500	19.000000
50%	100499.500000	2060.500000	10.355000	39.000000
75%	100749.250000	2843.000000	15.112500	59.000000
max	100999.000000	3598.000000	19.990000	77.000000

	Dropoff Community Area	Fare	Tips	Tolls	\
count	1000.000000	1000.000000	1000.000000	1000.000000	
mean	38.346000	28.320520	4.94992	1.515000	
std	22.469427	13.392485	2.87325	1.995936	
min	1.000000	4.520000	0.00000	0.000000	
25%	18.000000	17.355000	2.54750	0.000000	
50%	38.000000	28.100000	4.85500	0.000000	
75%	58.000000	39.517500	7.48000	2.500000	
max	77.000000	51.230000	9.98000	5.000000	

	Extras	Trip Total
count	1000.000000	1000.000000
mean	1.56900	36.354440
std	1.12628	13.775222
min	0.00000	5.900000
25%	1.00000	24.667500
50%	2.00000	36.930000
75%	3.00000	48.020000
max	3.00000	66.420000

```
df.duplicated().sum()
```

```
np.int64(0)
```

```
numeric_df = df.select_dtypes(include=['number'])
```

```
numeric_df.corr()
```

	Trip ID	Trip Seconds	Trip Miles	\
Trip ID	1.000000	0.011145	-0.009190	
Trip Seconds	0.011145	1.000000	0.006770	
Trip Miles	-0.009190	0.006770	1.000000	

Pickup Community Area	-0.022025	0.038086	-0.029371
Dropoff Community Area	-0.006709	0.007301	0.004200
Fare	-0.009189	0.006768	1.000000
Tips	-0.019017	0.021202	-0.046220
Tolls	-0.035953	0.050606	0.010815
Extras	0.027500	-0.031281	-0.004476
Trip Total	-0.015862	0.015777	0.963776

	Pickup Community Area	Dropoff Community Area
\		
Trip ID	-0.022025	-0.006709
Trip Seconds	0.038086	0.007301
Trip Miles	-0.029371	0.004200
Pickup Community Area	1.000000	-0.046253
Dropoff Community Area	-0.046253	1.000000
Fare	-0.029369	0.004212
Tips	0.021491	-0.020024
Tolls	0.025386	-0.036587
Extras	0.005064	-0.061265
Trip Total	-0.019978	-0.010392

	Fare	Tips	Tolls	Extras	Trip
Total					
Trip ID	-0.009189	-0.019017	-0.035953	0.027500	-
0.015862					
Trip Seconds	0.006768	0.021202	0.050606	-0.031281	
0.015777					
Trip Miles	1.000000	-0.046220	0.010815	-0.004476	
0.963776					
Pickup Community Area	-0.029369	0.021491	0.025386	0.005064	-
0.019978					
Dropoff Community Area	0.004212	-0.020024	-0.036587	-0.061265	-
0.010392					
Fare	1.000000	-0.046211	0.010811	-0.004491	
0.963776					
Tips	-0.046211	1.000000	0.024750	-0.038716	
0.164074					
Tolls	0.010811	0.024750	1.000000	-0.006472	
0.160037					
Extras	-0.004491	-0.038716	-0.006472	1.000000	

```

0.068382
Trip Total          0.963776  0.164074  0.160037  0.068382
1.000000

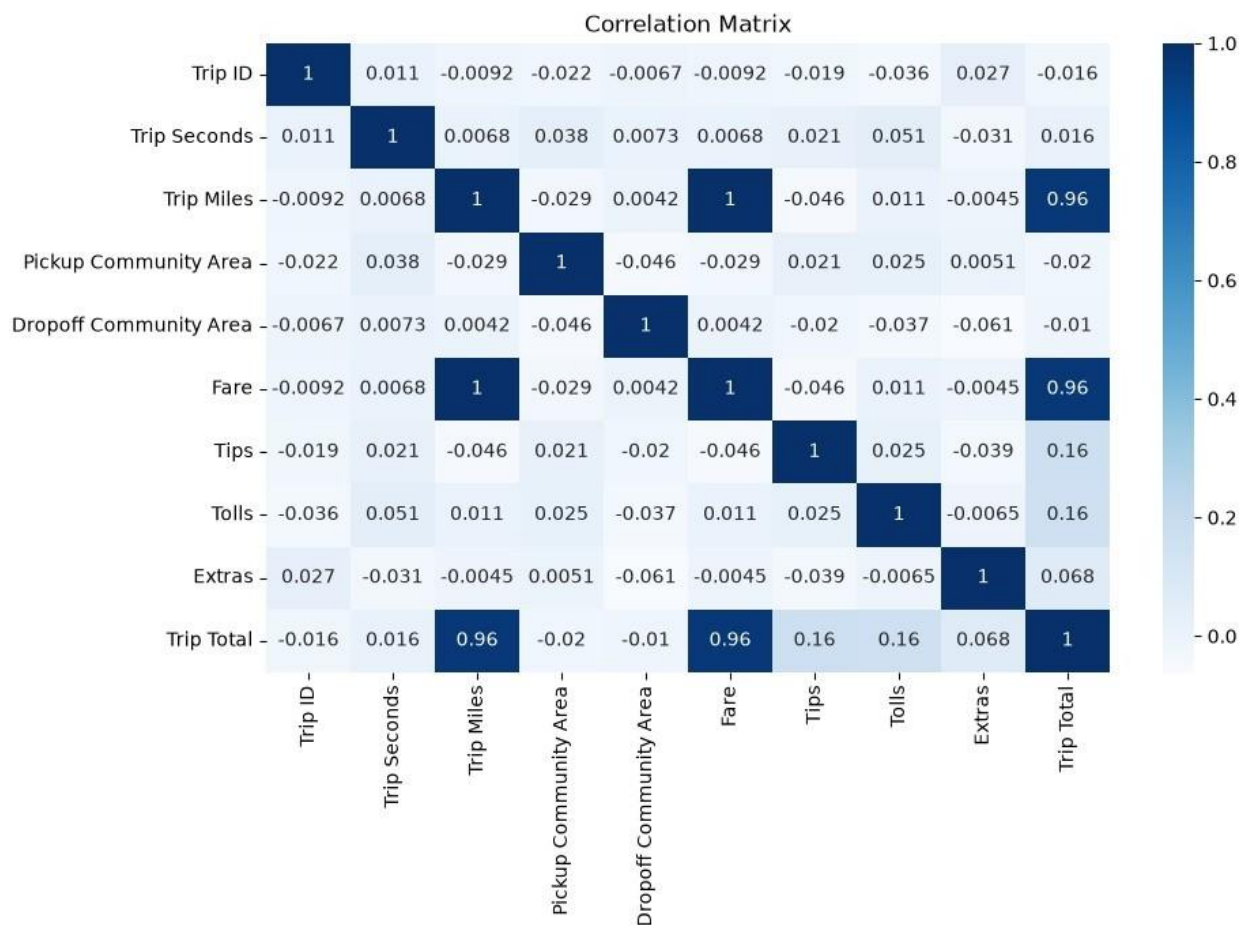
plt.figure(figsize=(10,6))

sns.heatmap(numeric_df.corr(),
            annot=True,
            cmap="Blues")

plt.title("Correlation Matrix")

plt.show()

```



```

plt.figure(figsize=(6,4))

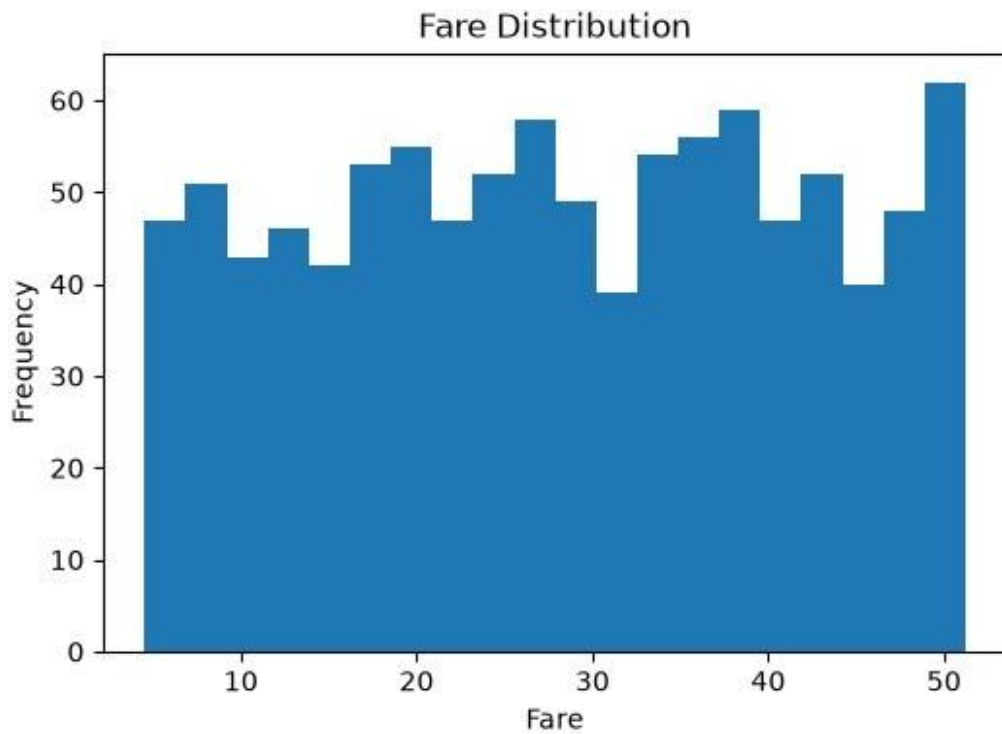
plt.hist(df["Fare"],bins=20)

plt.title("Fare Distribution")

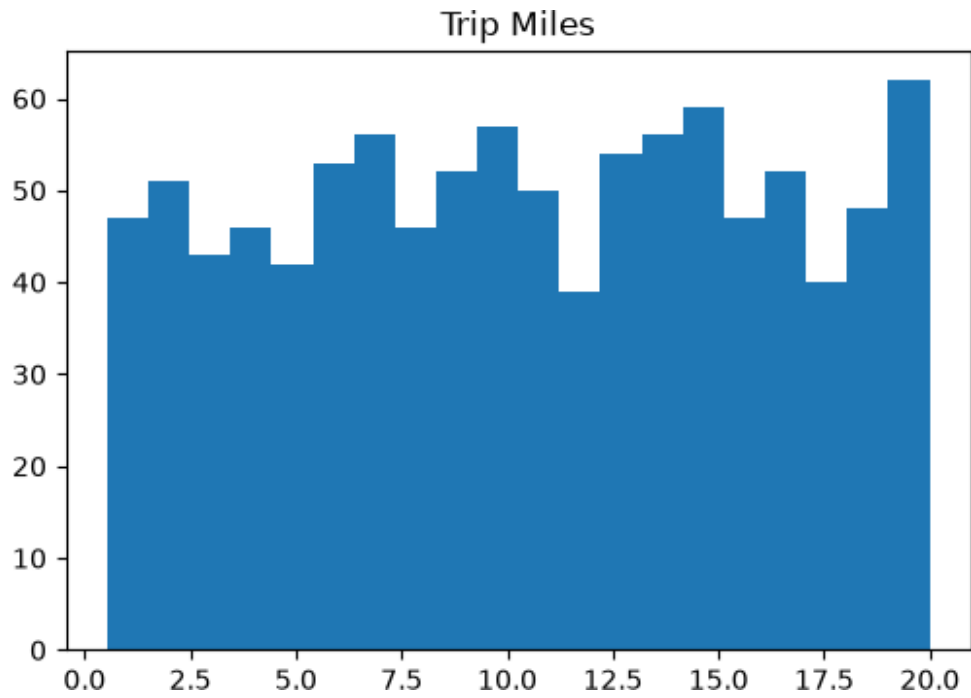
plt.xlabel("Fare")

```

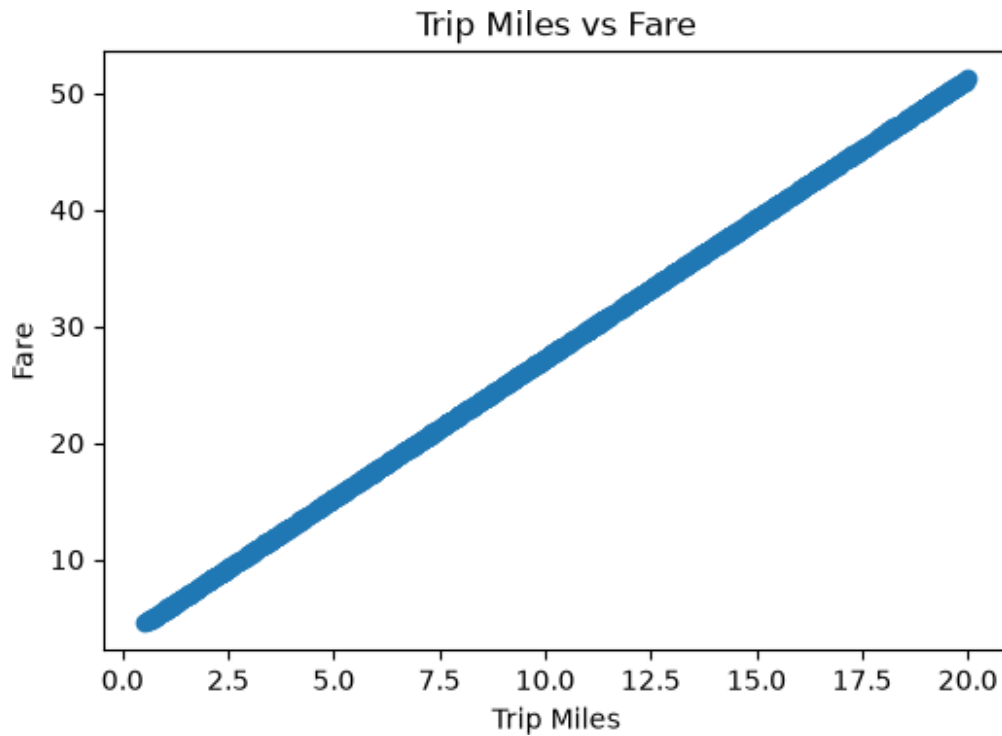
```
plt.ylabel("Frequency")  
plt.show()
```



```
plt.figure(figsize=(6,4))  
plt.hist(df["Trip Miles"],bins=20)  
plt.title("Trip Miles")  
plt.show()
```



```
plt.figure(figsize=(6,4))  
plt.scatter(df["Trip Miles"],  
            df["Fare"])  
plt.xlabel("Trip Miles")  
plt.ylabel("Fare")  
plt.title("Trip Miles vs Fare")  
plt.show()
```



```
X = df[['Trip Miles',
        'Trip Seconds',
        'Tips',
        'Tolls',
        'Extras']]
```

```
y = df['Fare']
```

```
X.head()
```

	Trip Miles	Trip Seconds	Tips	Tolls	Extras
0	0.99	756	2.75	0.0	1
1	4.76	683	5.05	0.0	1
2	15.30	326	1.60	2.5	2
3	2.39	1856	8.47	5.0	2
4	6.22	2561	6.29	0.0	1

```
y.head()
```

```
0    5.63
1   14.67
2   39.97
3    8.99
4   18.18
```

```
Name: Fare, dtype: float64
```

```
X_train,X_test,y_train,y_test=train_test_split(
    X,
```

```

    y,
    test_size=0.20,
    random_state=42
)

print(X_train.shape)

print(X_test.shape)

print(y_train.shape)

print(y_test.shape)

(800, 5)
(200, 5)
(800,)
(200,)

model = LinearRegression()

model.fit(X_train,y_train)

LinearRegression()

y_pred = model.predict(X_test)

mae = mean_absolute_error(y_test,y_pred)

print("Mean Absolute Error =",mae)

Mean Absolute Error = 0.0025140815069102816

mse = mean_squared_error(y_test,y_pred)

print("Mean Squared Error =",mse)

Mean Squared Error = 8.361545233724807e-06

rmse = np.sqrt(mse)

print("Root Mean Squared Error =",rmse)

Root Mean Squared Error = 0.0028916336617429267

r2 = r2_score(y_test,y_pred)

print("R2 Score =",r2)

R2 Score = 0.9999999500461441

r2 = r2_score(y_test,y_pred)

print("R2 Score =",r2)

```

```

R2 Score = 0.9999999500461441

print(model.intercept_)

3.2507366183733026

new_trip = pd.DataFrame({
    "Trip Miles": [8.5],
    "Trip Seconds": [1200],
    "Tips": [3],
    "Tolls": [0],
    "Extras": [2]
})

predicted_fare = model.predict(new_trip)

print("Predicted Fare = $", predicted_fare[0])

Predicted Fare = $ 23.65005379402034

results = pd.DataFrame({
    "Actual Fare": y_test,
    "Predicted Fare": y_pred
})

results.head()

```

	Actual Fare	Predicted Fare
521	45.03	45.033519
737	30.75	30.754014
740	21.47	21.466207
660	26.72	26.722202
411	25.55	25.546157

```

plt.figure(figsize=(6,5))

plt.scatter(y_test,y_pred)

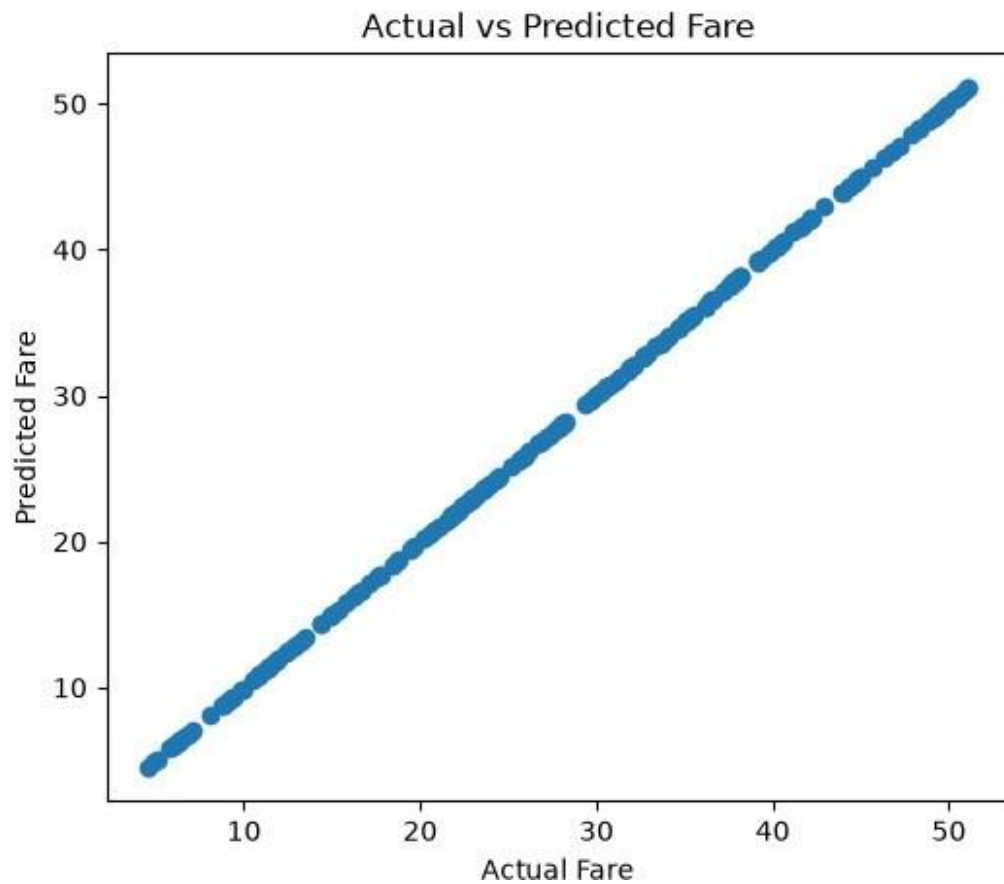
plt.xlabel("Actual Fare")

plt.ylabel("Predicted Fare")

plt.title("Actual vs Predicted Fare")

plt.show()

```



```
results.to_csv("Taxi_Fare_Predictions.csv", index=False)
print("Predictions saved successfully.")
Predictions saved successfully.
```